Email Spam Detection Progress Report

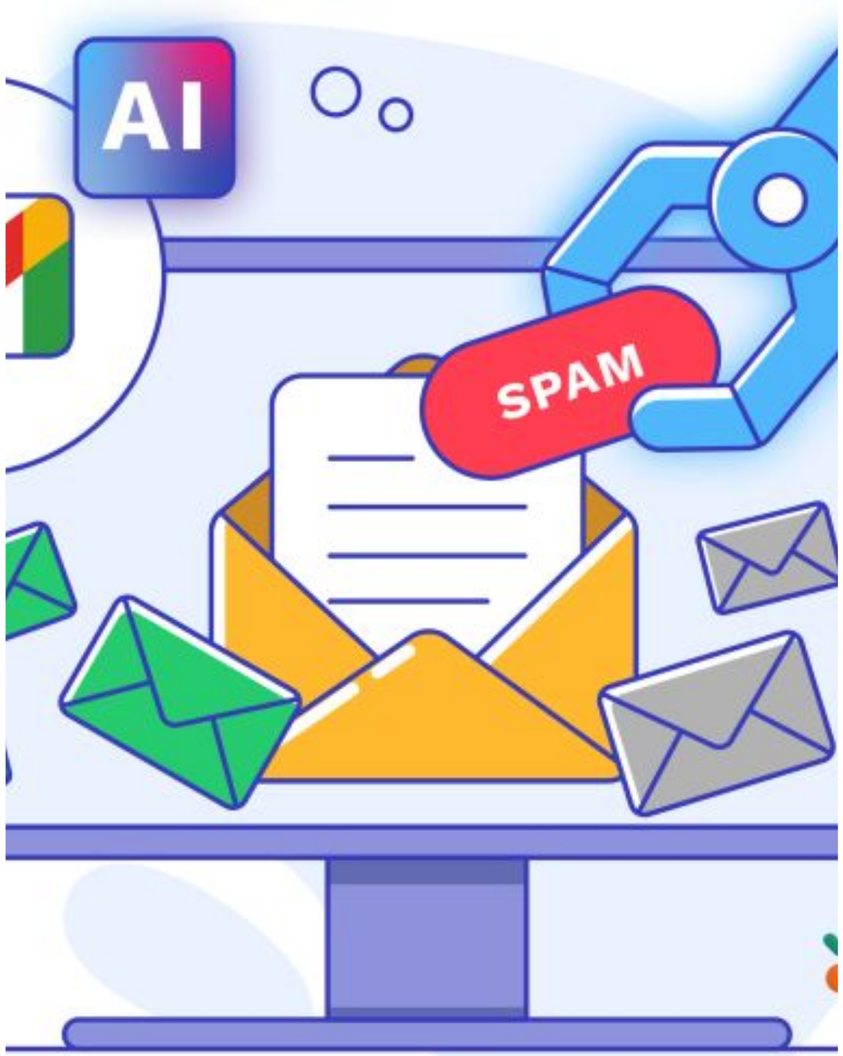
Group #7

- Emir Büçkün - 150119024
- Mehmet Sina Çağlar - 150123821
- Fatma Melisa Küçük - 150119916
- İrem Kubalas - 150119721
- Ayşenur Tüfekçi - 150119699

Instructor: Assoc. Prof. Murat Can Ganiz

December 16, 2024

Last Edited:
December 23, 2024



Introduction & Goals

Spam emails clutter inboxes and often pose security threats.

Our goal was to create a machine learning-based spam detection system to address this issue.

This system helps users filter emails more effectively and focus on important communications.

1

DATA PREPROCESSING & EDA

Understand the data and conduct exploratory data analysis

2

MODEL DEVELOPMENT

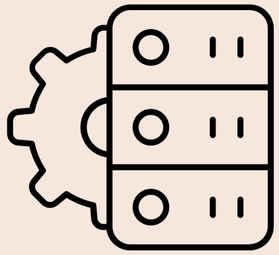
Implement initial models and train models using the training data

3

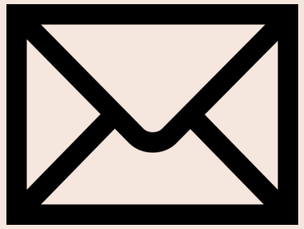
MODEL EVALUATION AND OPTIMIZATION

Evaluate model performance using test data and experiment with different algorithms and hyperparameters

The Dataset



We used the **Enron Spam Dataset** from Kaggle.

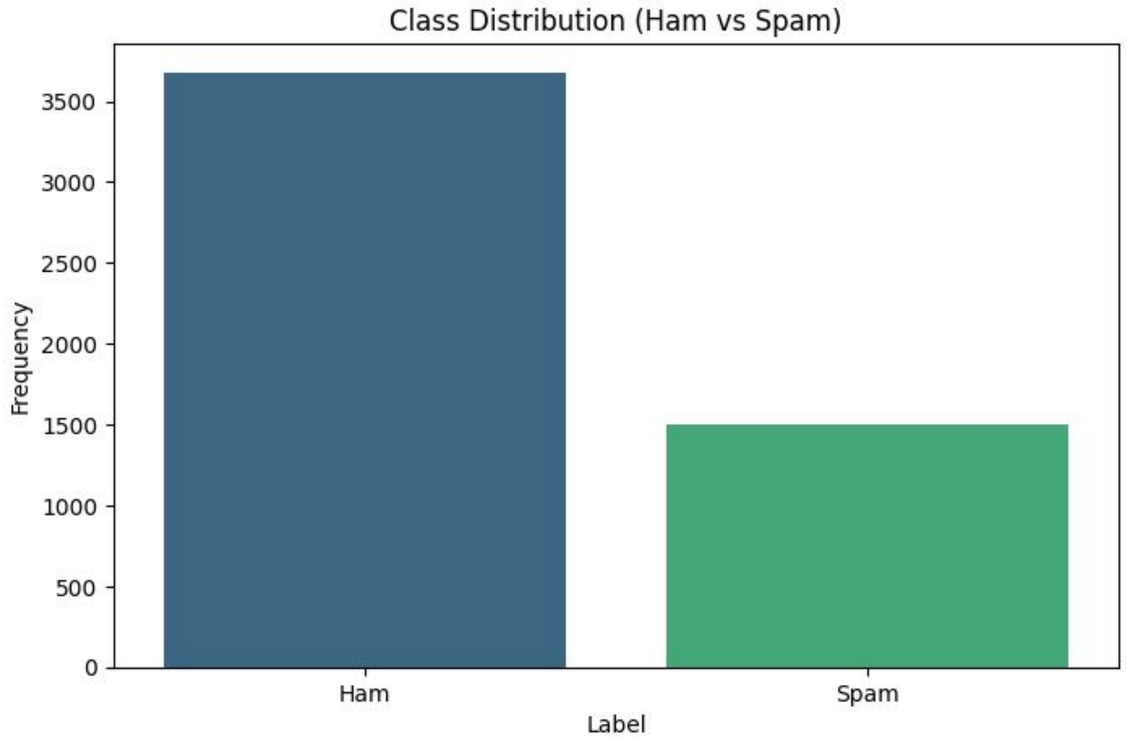


It included **5,172** messages labeled as "Ham" (legitimate) and "Spam."

One challenge we faced early on was the dataset being imbalanced, with **far more legitimate emails than spam.**

Enron Spam Dataset - Kaggle

The distribution of emails in the dataset: approximately 3,500 Ham (legitimate) emails shown in blue versus 1,500 Spam emails shown in green.



1

Goal #1

Data Preprocessing & EDA

DONE

COLLABORATORS

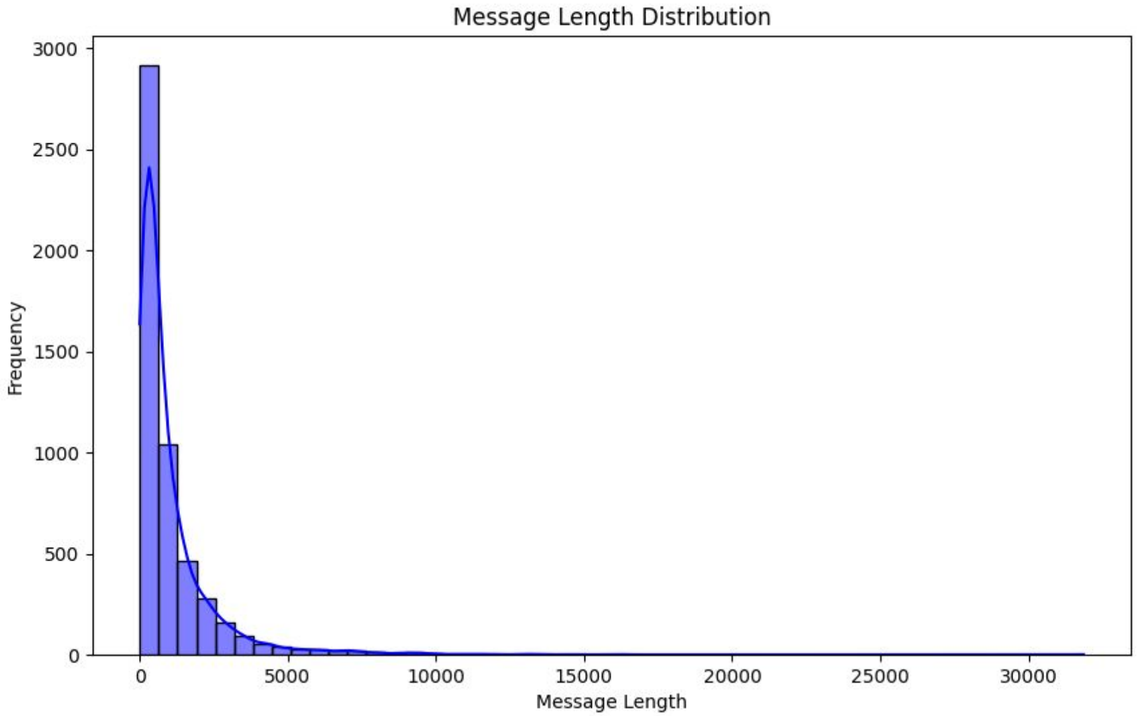
- EMİR BÜÇKÜN
- İREM KUBALAS
- MEHMET SİNA ÇAĞLAR

PROGRESS SUMMARY

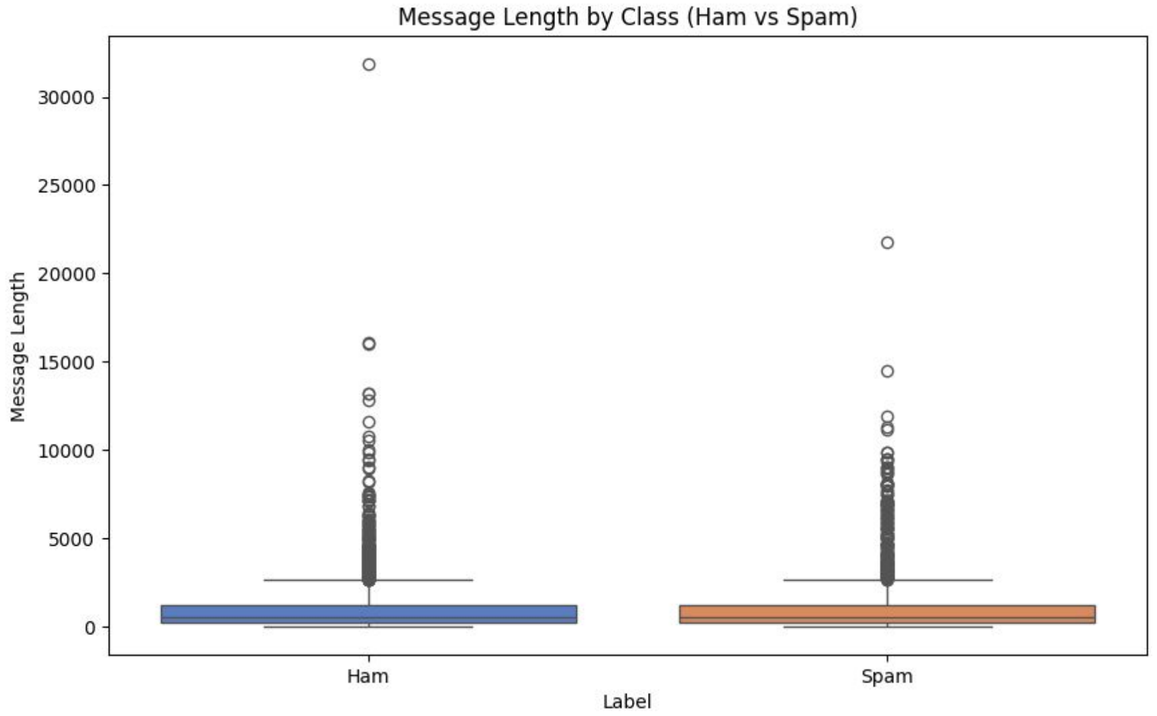
- Data Preprocessing:
  - Removed special characters, converted to lowercase
  - Eliminated stopwords using NLTK library
  - Created clean message column and saved to CSV
- Dataset is now properly formatted and labeled
- Performed data analysis:
  - Generated message length statistics and distributions
  - Analyzed class distribution between ham and spam
  - Created visualizations using matplotlib and seaborn
- Implemented feature engineering:
  - Added message length and word count features
  - Calculated punctuation counts and capital letter ratios
  - Identified spam keywords using TF-IDF vectorization
  - Created spam keyword frequency features

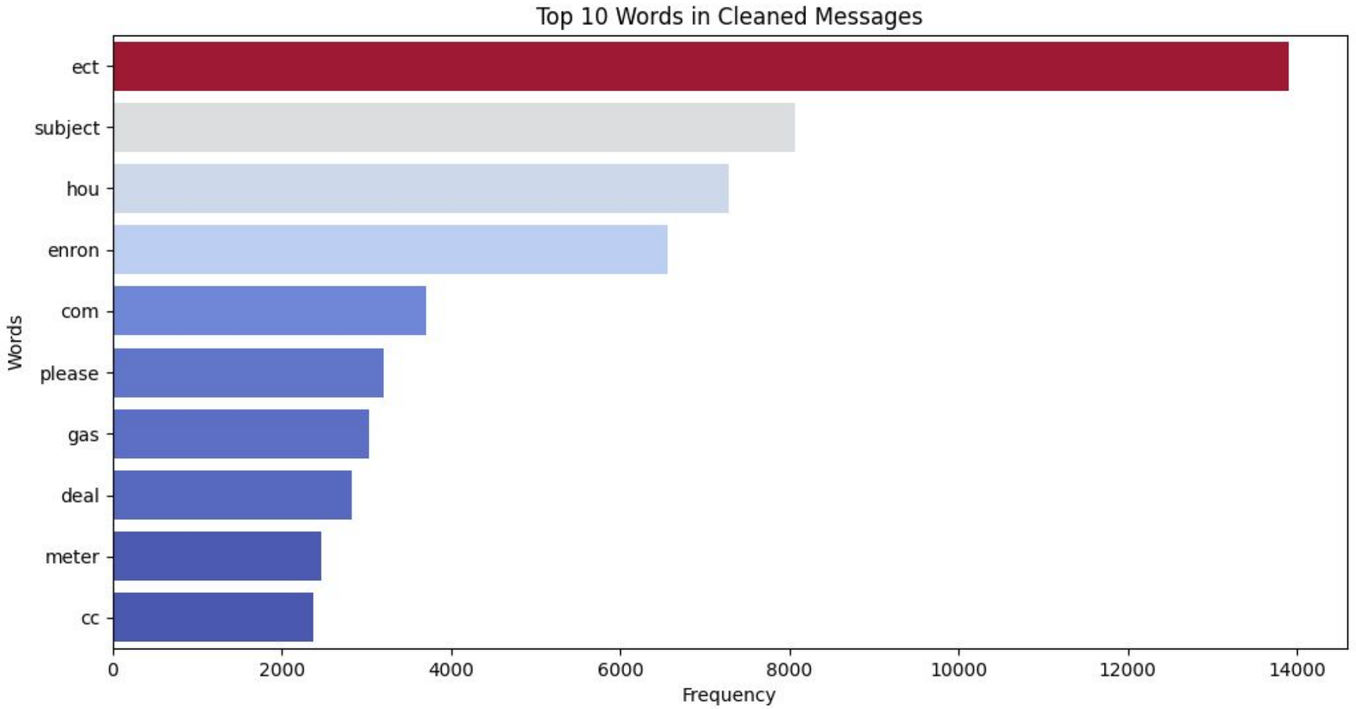
A histogram displaying the distribution of message lengths in the email dataset.

There's a peak frequency of around 2,800 characters at the shorter lengths, and the distribution gradually tails off towards longer message lengths up to 30,000 characters.



Boxplot visualization shows the distribution of message lengths for ham (legitimate) and spam emails. The plot reveals that spam messages tend to be shorter in length compared to ham messages.





The bar chart displays the top 10 most frequently occurring words in the cleaned email messages. The word "ect" appears most frequently with around 14,000 occurrences, followed by "subject" and "hou" with approximately 7,000-8,000 occurrences.

Engineered Features:

	Message_Length	Word_Count	Punctuation_Count	Capital_Letter_Ratio	Spam_Keyword_Count
0	2597	247	147	0.000385	3
1	549	30	38	0.001821	1
2	103	11	8	0.009709	1
3	530	44	81	0.001887	1
4	1391	139	136	0.000719	3

Engineered Features

- Message Length: Length of the message text.
- Word Count: Number of words in the cleaned message.
- Punctuation Count: Total punctuation marks in the original message.
- Capital Letter Ratio: Proportion of capital letters to total characters.
- Spam Keyword Count: Spam keywords identified and calculated their counts

2

Goal #2

Model Development

DONE

COLLABORATORS

- EMİR BÜÇKÜN
- İREM KUBALAS
- MEHMET SİNA ÇAĞLAR
- AYŞENUR TÜFEKÇİ
- FATMA MELİSA KÜÇÜK

PROGRESS SUMMARY

- The dataset was split into training and test sets with 80% training and 20% testing using train_test_split.

- Initial Model: **Random Forest**
  - It is chosen as initial model because of its strong reputation for classification tasks and handling feature importance effectively.
  - A baseline accuracy of **86%**, which set a standard for evaluating other models.
- Exploring Other Models:
  - Trained several additional models:
    - Logistic Regression
    - Naive Bayes
    - K-Nearest Neighbors

How the Models Performed?

Random Forest

300 decision trees and using a random state of 42.

Accuracy: 86%

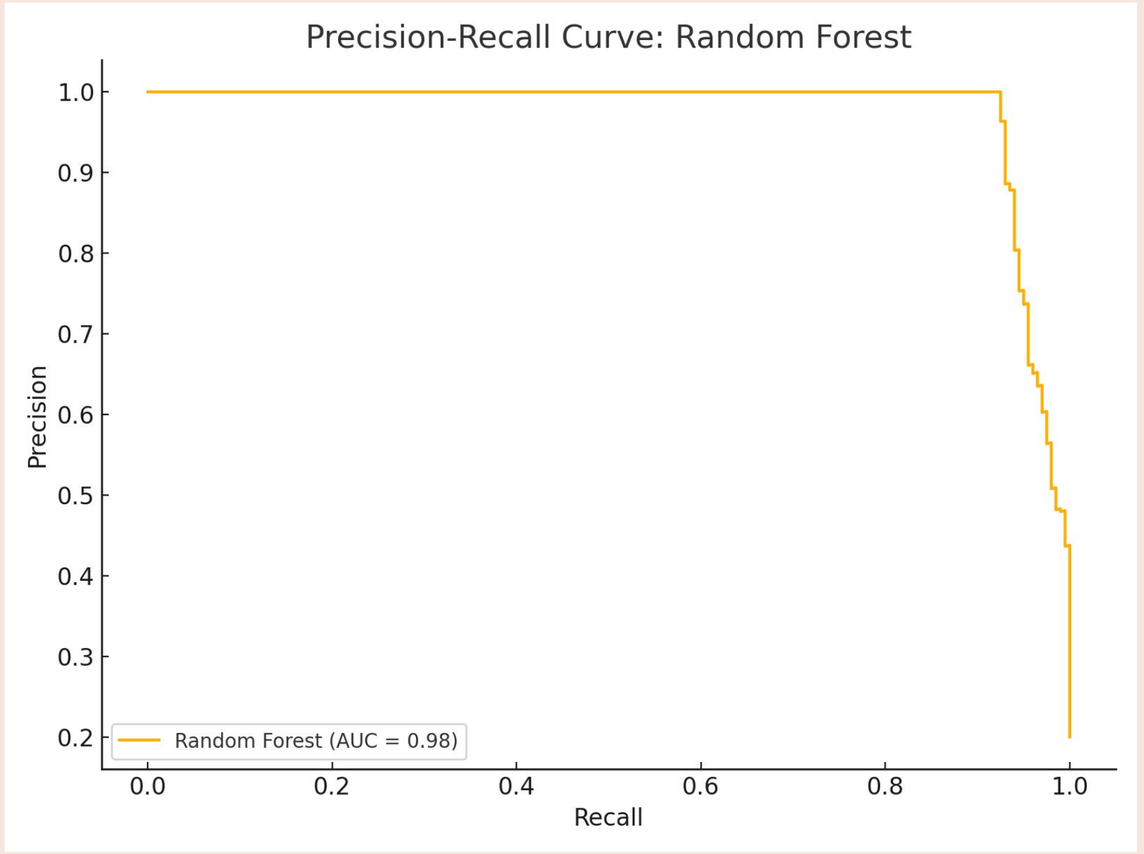
Precision: 80%

Recall: 70%

F1-Score: 75%

Random Forest emerged as the best performer, striking a balance between correctly identifying spam (high recall) and avoiding false positives (high precision).

The model's ability to handle a variety of features (e.g., message length, punctuation count, capital letter ratio) made it robust and effective.



How the Models Performed?

Logistic Regression

The default solver and a random state of 42

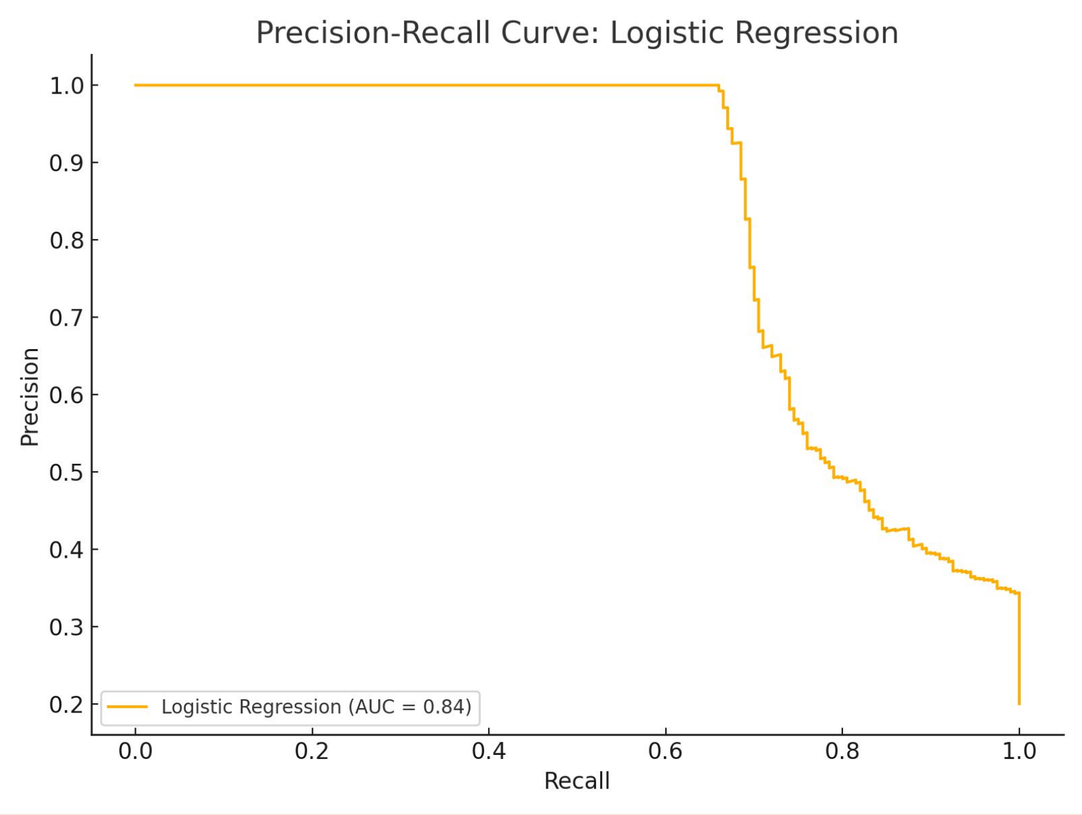
Accuracy: 79%

Precision: 76%

Recall: 43%

F1-Score: 55%

While Logistic Regression achieved decent precision, it struggled with recall, meaning it missed many spam messages. This was partly due to the class imbalance in the dataset.



How the Models Performed?

Multinomial Naive Bayes

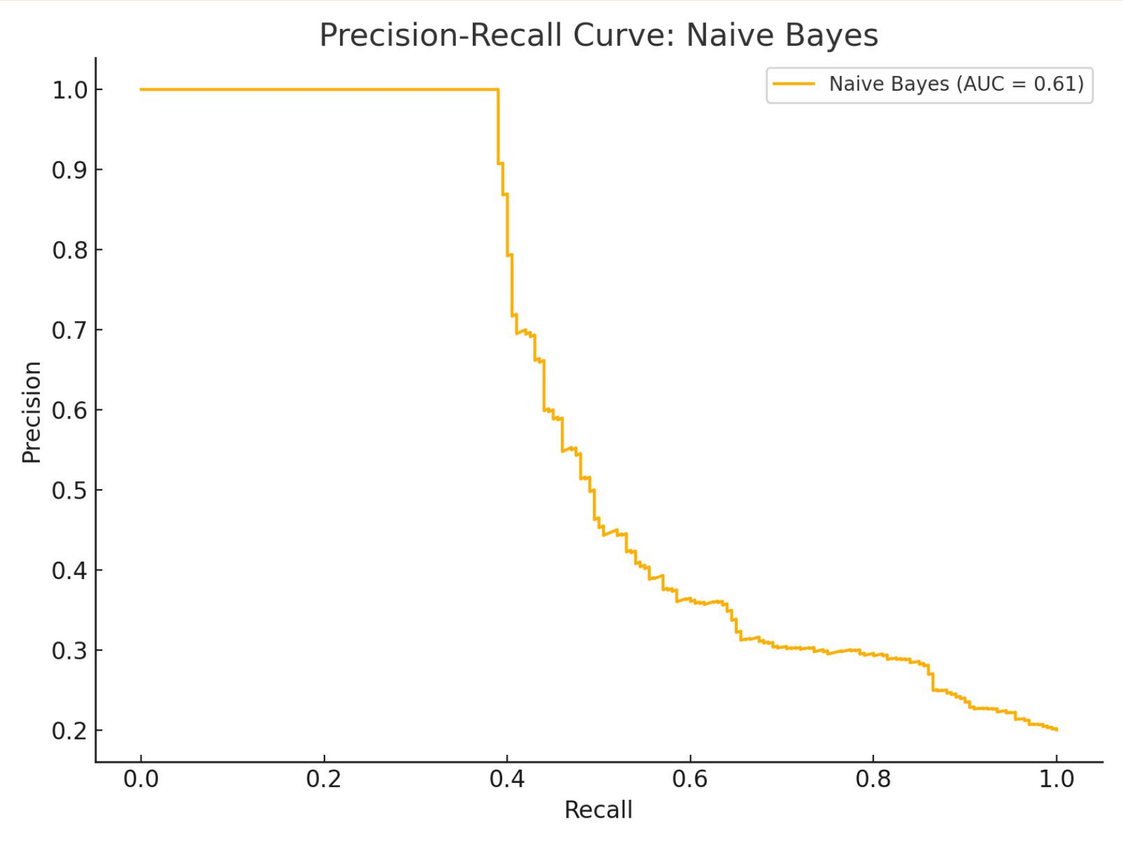
Accuracy: 60%

Precision: 39%

Recall: 65%

F1-Score: 48%

Naive Bayes struggled with precision, often misclassifying legitimate emails as spam. However, its recall was better than Logistic Regression, meaning it caught more spam messages overall.



How the Models Performed?

k-Nearest Neighbors (k-NN)

The K-Nearest Neighbors model was built,

with 5 neighbors.

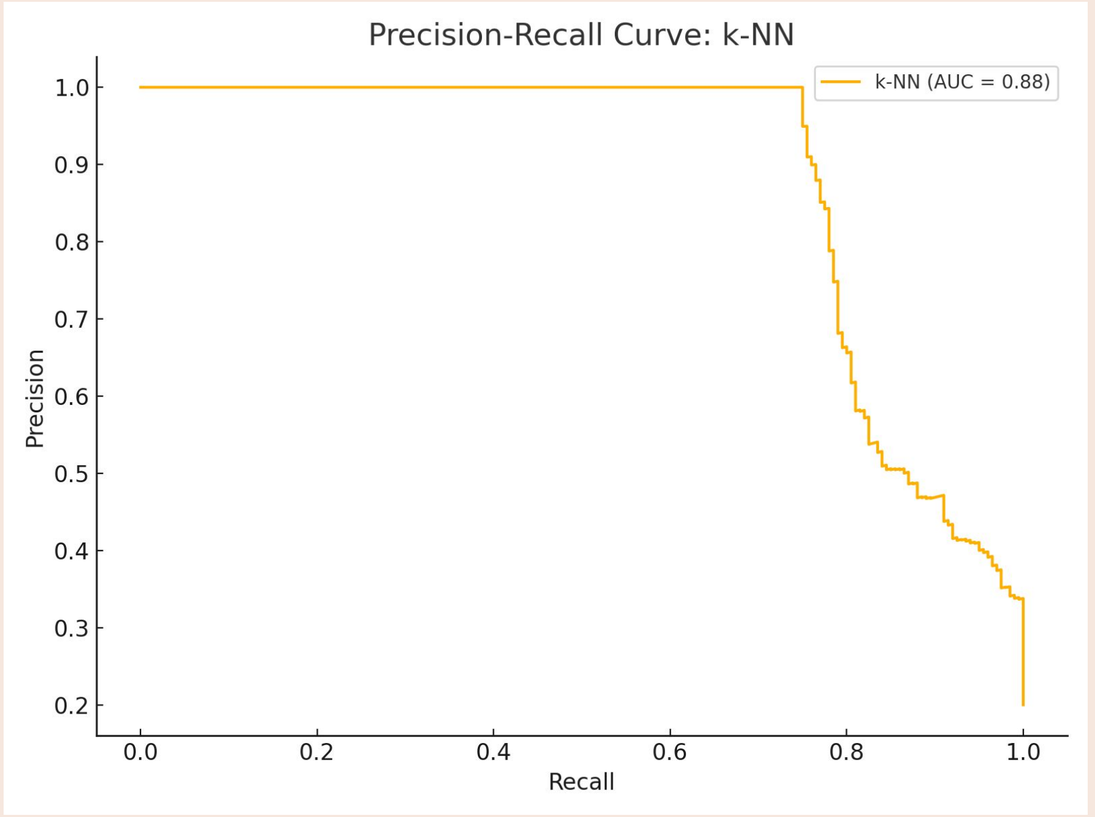
Accuracy: 78%

Precision: 66%

Recall: 46%

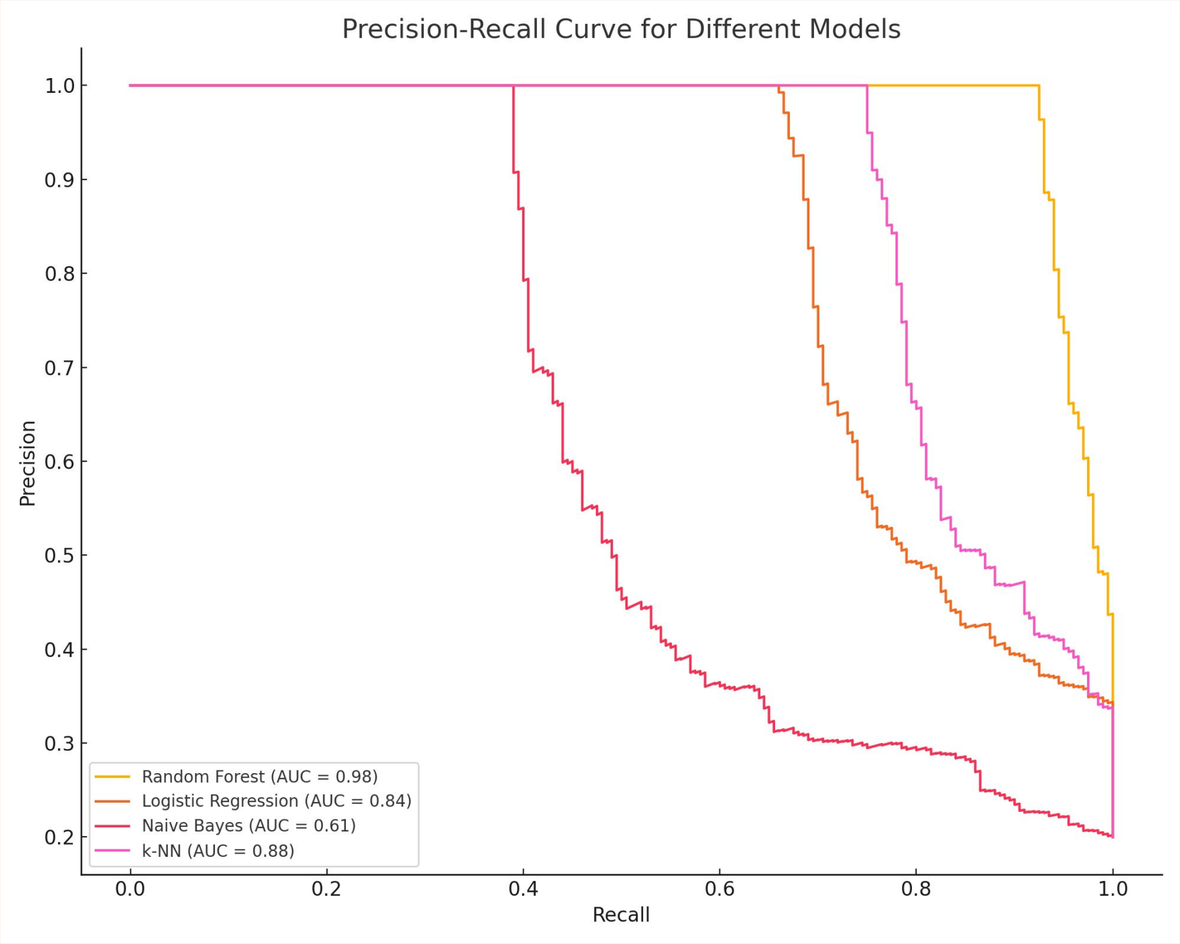
F1-Score: 55%

k-NN showed balanced performance but lagged behind Random Forest in terms of overall accuracy and precision.



Observations

- **Random Forest** proved to be the most reliable model due to its ability to balance precision and recall effectively.
- **Class imbalance** negatively impacted models like Logistic Regression and Naive Bayes, highlighting the importance of techniques like oversampling or weighting classes.
- The relatively low recall for Logistic Regression indicates that it missed a significant proportion of actual spam messages.
- Naive Bayes' low precision suggests that it often misclassified legitimate emails as spam, which could annoy users in real-world applications.



Metric/Model	Random Forest	Logistic Regression	Naive Bayes	k-NN
Accuracy	86%	79%	60%	78%
Precision	80%	76%	39%	66%
Recall	70%	43%	65%	46%
F1-Score	75%	55%	48%	55%

3 **Goal #3**

MODEL EVALUATION AND OPTIMIZATION

DONE

COLLABORATORS

- EMİR BÜÇKÜN
- İREM KUBALAS
- MEHMET SİNA ÇAĞLAR
- AYŞENUR TÜFEKÇİ
- FATMA MELİSA KÜÇÜK

PROGRESS SUMMARY

- Applied **Grid Search** and **Randomized Search**:
  - Improved Random Forest to achieve 87% accuracy and better recall.
  - Tuned Logistic Regression, Naive Bayes, and k-NN for slight improvements.
- Compared optimized models based on accuracy, precision, recall, and F1-score:
  - Random Forest consistently outperformed others.
  - Naive Bayes showed minimal improvement even after optimization.
  - k-NN remained sensitive to high-dimensional data.

Optimized Model Performation

Random Forest

Grid Search:

Accuracy 86%,

Precision (Spam) 82%,

Recall 64%,

F1-Score 72%.

Randomized Search:

Accuracy 87%,

Precision (Spam) 82%,

Recall 69%,

F1-Score 75%.

Best performer with significant improvement from optimization.

Error analysis:

- **False Positives**: Some ham messages were misclassified as spam, potentially due to overlapping feature distributions.
- **False Negatives**: Spam messages with subtle patterns were missed, suggesting a need for additional feature engineering.

Optimized Model Performation

Logistic Regression

Grid Search:

Accuracy 80%,

Precision (Spam) 78%,

Recall 44%,

F1-Score 56%.

Despite optimization, Logistic Regression struggled with recall, likely due to class imbalance.

Error analysis:

- **Low Recall**: The model struggled to identify spam messages, likely due to the imbalanced dataset and linear assumptions.

Optimized Model Performation

Naive Bayes

Grid Search:

Accuracy 78%,

Precision (Spam) 39%,

Recall 65%,

F1-Score 48%.

Naive Bayes remained limited by its assumption of feature independence.

Error analysis:

- **Feature Independence Assumption**: Failed to capture complex dependencies, leading to poor overall performance.

Optimized Model Performation

k-Nearest Neighbors (k-NN)

Grid Search:

Accuracy 79%,

Precision (Spam) 72%,

Recall 47%,

F1-Score 57%.

While improvements were minor, k-NN's performance was affected by high-dimensional data.

Error analysis:

- **Dimensionality Sensitivity**: Performance degradation occurred due to high-dimensional data and class imbalance.

Challenges & Lessons

Class imbalance made it harder for some models to identify spam correctly.

Preprocessing the text (e.g., cleaning and removing stopwords) was a time-consuming but essential step.

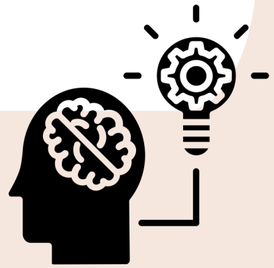
Some models needed significant tuning to improve recall and precision without overfitting. (hyperparameter tuning)

Identifying impactful features, like message length or spam keywords, required trial and error.

We realized how crucial it is to fully understand the dataset before building models.

Proper handling of class imbalance can make or break model performance.

Iterating on feature engineering is just as important as selecting the right model.



Conclusion

We built a reliable spam classifier, with Random Forest emerging as the best-performing model.

Despite challenges, this project taught us the importance of evaluation metrics and thorough analysis.

There's still room to improve—especially in tackling class imbalance and exploring advanced methods.

THE WINNER IS

RANDOM FOREST

Thank you!

Do you have any questions?